

```
!pip install sqlalchemy pymysql pandas
```

```
Requirement already satisfied: sqlalchemy in c:\programdata\anaconda3\lib\site-packages (2.0.43)
```

```
Collecting pymysql
```

```
  Downloading pymysql-1.2.0-py3-none-any.whl.metadata (4.3 kB)
```

```
Requirement already satisfied: pandas in c:\programdata\anaconda3\lib\site-packages (2.3.3)
```

```
Requirement already satisfied: greenlet>=1 in c:\programdata\anaconda3\lib\site-packages (from sqlalchemy) (3.2.4)
```

```
Requirement already satisfied: typing-extensions>=4.6.0 in c:\programdata\anaconda3\lib\site-packages (from sqlalchemy) (4.15.0)
```

```
Requirement already satisfied: numpy>=1.26.0 in c:\programdata\anaconda3\lib\site-packages (from pandas) (2.3.5)
```

```
Requirement already satisfied: python-dateutil>=2.8.2 in c:\programdata\anaconda3\lib\site-packages (from pandas) (2.9.0.post0)
```

```
Requirement already satisfied: pytz>=2020.1 in c:\programdata\anaconda3\lib\site-packages (from pandas) (2025.2)
```

```
Requirement already satisfied: tzdata>=2022.7 in c:\programdata\anaconda3\lib\site-packages (from pandas) (2025.2)
```

```
Requirement already satisfied: six>=1.5 in c:\programdata\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

```
Downloading pymysql-1.2.0-py3-none-any.whl (45 kB)
```

```
Installing collected packages: pymysql
```

```
Successfully installed pymysql-1.2.0
```

```
import pandas as pd
from sqlalchemy import create_engine
```

```
# Format: mysql+pymysql://username:password@host:port/database_name
```

```
engine =
```

```
create_engine("mysql+pymysql://root:Sleepingdogs11@localhost:3306/black_friday_project")
```

```
df = pd.read_sql("SELECT * FROM black_friday_data", engine)
```

```
print(df.shape)
```

```
df.head()
```

```
(550068, 12)
```

|   | User_ID | Product_ID | Gender | Age  | Occupation | City_Category | \ |
|---|---------|------------|--------|------|------------|---------------|---|
| 0 | 1000001 | P00000142  | F      | 0-17 | 10         | A             |   |
| 1 | 1000001 | P00004842  | F      | 0-17 | 10         | A             |   |
| 2 | 1000001 | P00025442  | F      | 0-17 | 10         | A             |   |
| 3 | 1000001 | P00051442  | F      | 0-17 | 10         | A             |   |
| 4 | 1000001 | P00051842  | F      | 0-17 | 10         | A             |   |

|   | Stay_In_Current_City_Years | Marital_Status | Product_Category_1 | \ |
|---|----------------------------|----------------|--------------------|---|
| 0 | 2                          | 0              | 3                  |   |
| 1 | 2                          | 0              | 3                  |   |
| 2 | 2                          | 0              | 1                  |   |

|   |   |   |   |
|---|---|---|---|
| 3 | 2 | 0 | 8 |
| 4 | 2 | 0 | 4 |

|   | Product_Category_2 | Product_Category_3 | Purchase |
|---|--------------------|--------------------|----------|
| 0 | 4.0                | 5.0                | 13650    |
| 1 | 4.0                | 12.0               | 13645    |
| 2 | 2.0                | 9.0                | 15416    |
| 3 | 17.0               | NaN                | 9938     |
| 4 | 8.0                | NaN                | 2849     |

*# Treat missing values in Product\_Category\_2 and Product\_Category\_3*

```
df['Product_Category_2'] = df['Product_Category_2'].fillna(0)
```

```
df['Product_Category_3'] = df['Product_Category_3'].fillna(0)
```

```
df.isnull().sum()
```

```
User_ID      0
Product_ID    0
Gender        0
Age           0
Occupation    0
City_Category 0
Stay_In_Current_City_Years 0
Marital_Status 0
Product_Category_1 0
Product_Category_2 0
Product_Category_3 0
Purchase      0
dtype: int64
```

*# Customer Lifetime Value (CLV) = Total Purchase per User / Number of Transactions*

```
customer_agg = df.groupby('User_ID').agg(
    Total_Purchase=('Purchase', 'sum'),
    Num_Transactions=('Purchase', 'count')
).reset_index()
```

```
customer_agg['CLV'] = customer_agg['Total_Purchase'] /
customer_agg['Num_Transactions']
```

*# Category Breadth = Distinct product categories purchased per customer*

```
category_breadth = df.groupby('User_ID').apply(
    lambda x:
pd.unique(x[['Product_Category_1', 'Product_Category_2', 'Product_Category_3']].values.ravel()))
).apply(lambda x: len(x[x != 0]))
category_breadth =
category_breadth.reset_index(name='Category_Breadth')
```

*# Merge into customer\_agg*

```
customer_agg = customer_agg.merge(category_breadth, on='User_ID')

# City Loyalty Index = Avg Purchase x Stay Years
city_stay_map =
df[['User_ID', 'Stay_In_Current_City_Years']].drop_duplicates()
city_stay_map['Stay_In_Current_City_Years'] =
city_stay_map['Stay_In_Current_City_Years'].replace('4+', '4').astype(int)
customer_agg = customer_agg.merge(city_stay_map, on='User_ID')
customer_agg['Avg_Purchase'] = customer_agg['Total_Purchase'] /
customer_agg['Num_Transactions']
customer_agg['City_Loyalty_Index'] = customer_agg['Avg_Purchase'] *
customer_agg['Stay_In_Current_City_Years']

customer_agg.head()
```

C:\Users\TATHAGAT MUKHERJEE\AppData\Local\Temp\ipykernel\_2088\626588663.py:9: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```
category_breadth = df.groupby('User_ID').apply(
```

|                    | User_ID | Total_Purchase | Num_Transactions | CLV          |
|--------------------|---------|----------------|------------------|--------------|
| Category Breadth \ |         |                |                  |              |
| 0                  | 1000001 | 334093         | 35               | 9545.514286  |
| 14                 |         |                |                  |              |
| 1                  | 1000002 | 810472         | 77               | 10525.610390 |
| 15                 |         |                |                  |              |
| 2                  | 1000003 | 341635         | 29               | 11780.517241 |
| 11                 |         |                |                  |              |
| 3                  | 1000004 | 206468         | 14               | 14747.714286 |
| 11                 |         |                |                  |              |
| 4                  | 1000005 | 821001         | 106              | 7745.292453  |
| 16                 |         |                |                  |              |

|   | Stay_In_Current_City_Years | Avg_Purchase | City_Loyalty_Index |
|---|----------------------------|--------------|--------------------|
| 0 | 2                          | 9545.514286  | 19091.028571       |
| 1 | 4                          | 10525.610390 | 42102.441558       |
| 2 | 3                          | 11780.517241 | 35341.551724       |
| 3 | 2                          | 14747.714286 | 29495.428571       |
| 4 | 1                          | 7745.292453  | 7745.292453        |

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Histogram: purchase distribution
plt.figure(figsize=(8,5))
```

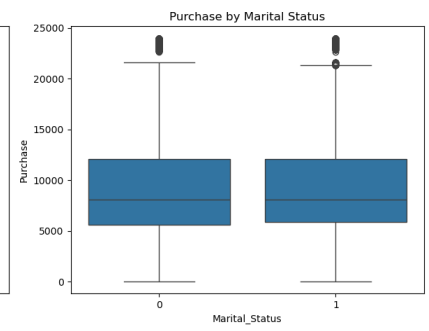
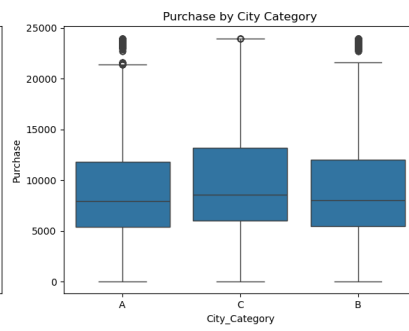
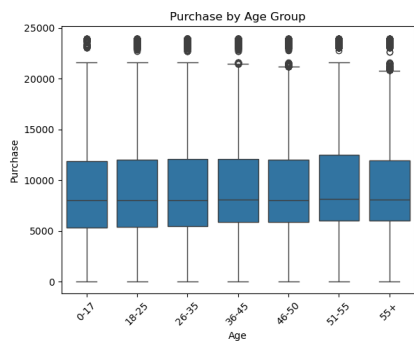
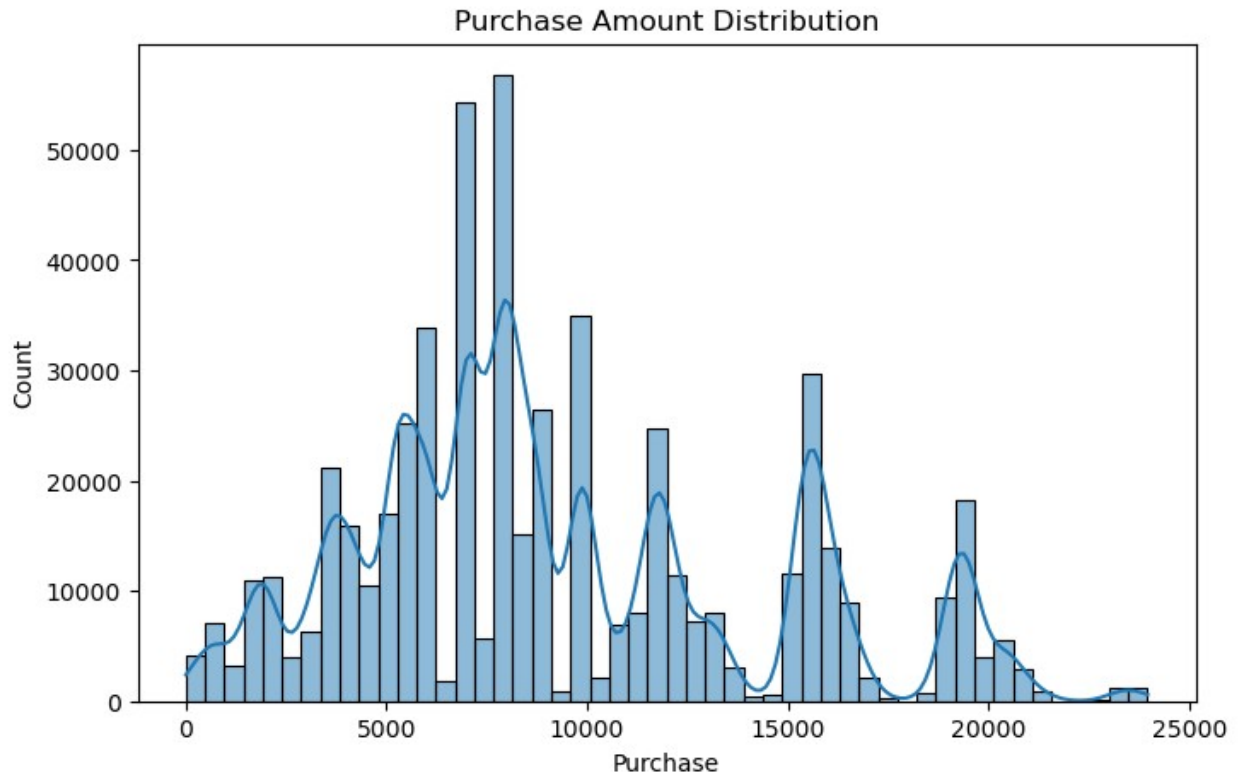
```
sns.histplot(df['Purchase'], bins=50, kde=True)
plt.title('Purchase Amount Distribution')
plt.show()

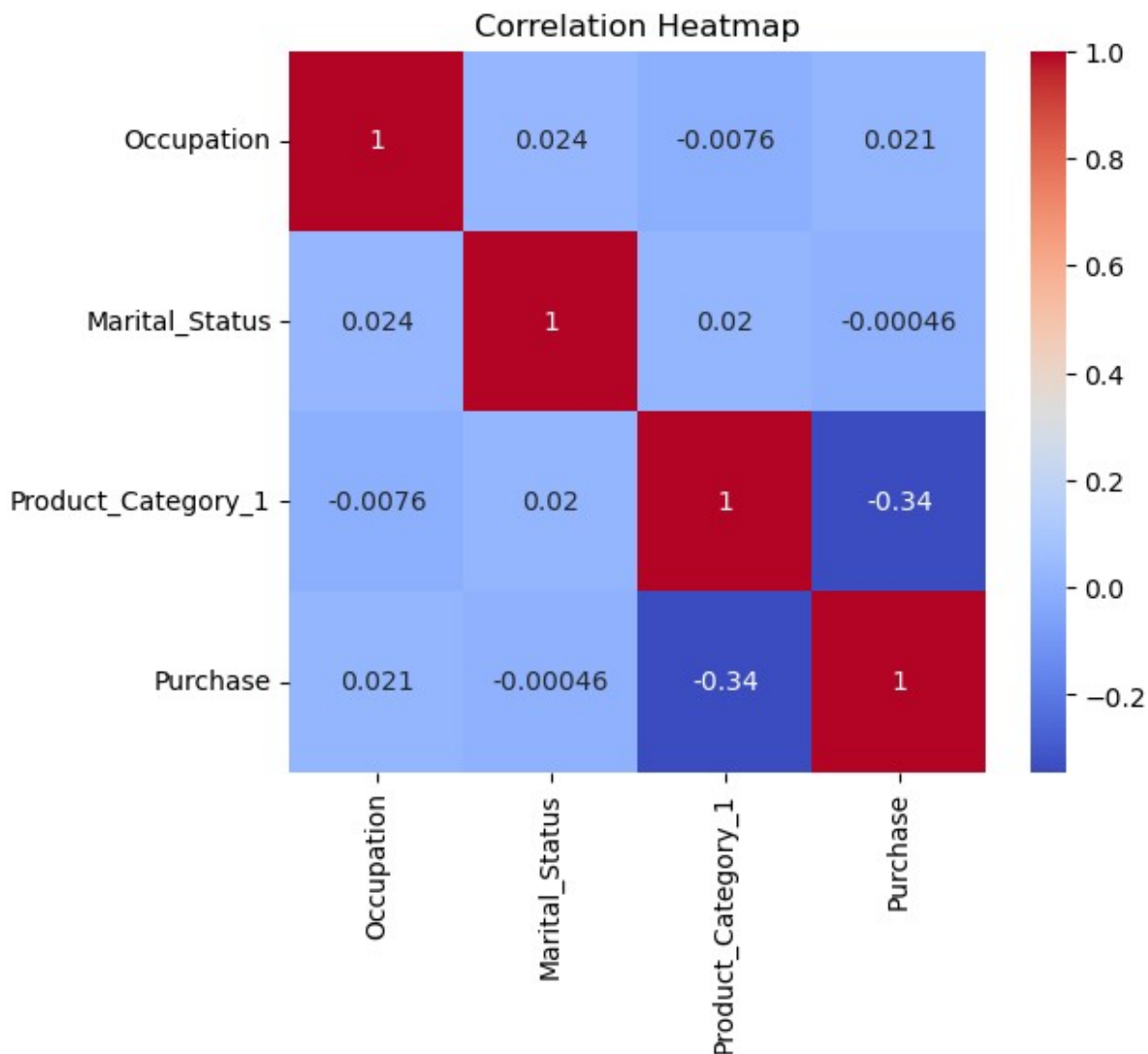
# Boxplots: by age group, city, marital status
fig, axes = plt.subplots(1, 3, figsize=(18,5))
sns.boxplot(data=df, x='Age', y='Purchase',
order=sorted(df['Age'].unique()), ax=axes[0])
axes[0].set_title('Purchase by Age Group')
axes[0].tick_params(axis='x', rotation=45)

sns.boxplot(data=df, x='City_Category', y='Purchase', ax=axes[1])
axes[1].set_title('Purchase by City Category')

sns.boxplot(data=df, x='Marital_Status', y='Purchase', ax=axes[2])
axes[2].set_title('Purchase by Marital Status')
plt.tight_layout()
plt.show()

# Heatmap: correlations between demographics and purchase
corr_df =
df[['Occupation', 'Marital_Status', 'Product_Category_1', 'Purchase']].co
rr()
plt.figure(figsize=(6,5))
sns.heatmap(corr_df, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```





```
from scipy import stats

# T-test: compare average purchase by Gender
male = df[df['Gender'] == 'M']['Purchase']
female = df[df['Gender'] == 'F']['Purchase']

t_stat, p_val = stats.ttest_ind(male, female)
print("T-test: Purchase by Gender")
print(f"T-statistic: {t_stat:.4f}, P-value: {p_val:.4e}")
print(f"Male avg: {male.mean():.2f}, Female avg: {female.mean():.2f}")

T-test: Purchase by Gender
T-statistic: 44.8380, P-value: 0.0000e+00
Male avg: 9437.53, Female avg: 8734.57

# T-test: compare average purchase by Marital Status
single = df[df['Marital_Status'] == 0]['Purchase']
```

```

married = df[df['Marital_Status'] == 1]['Purchase']

t_stat2, p_val2 = stats.ttest_ind(single, married)
print("T-test: Purchase by Marital Status")
print(f"T-statistic: {t_stat2:.4f}, P-value: {p_val2:.4e}")
print(f"Single avg: {single.mean():.2f}, Married avg: {married.mean():.2f}")

T-test: Purchase by Marital Status
T-statistic: 0.3437, P-value: 7.3109e-01
Single avg: 9265.91, Married avg: 9261.17

# ANOVA: Purchase across Age groups
age_groups = [df[df['Age'] == a]['Purchase'] for a in df['Age'].unique()]
f_stat, p_val3 = stats.f_oneway(*age_groups)
print("ANOVA: Purchase across Age Groups")
print(f"F-statistic: {f_stat:.4f}, P-value: {p_val3:.4e}")

# ANOVA: Purchase across City Categories
city_groups = [df[df['City_Category'] == c]['Purchase'] for c in df['City_Category'].unique()]
f_stat2, p_val4 = stats.f_oneway(*city_groups)
print("\nANOVA: Purchase across City Categories")
print(f"F-statistic: {f_stat2:.4f}, P-value: {p_val4:.4e}")

ANOVA: Purchase across Age Groups
F-statistic: 40.5758, P-value: 1.0536e-49

ANOVA: Purchase across City Categories
F-statistic: 1130.7460, P-value: 0.0000e+00

# Correlation: Occupation, City Tenure, Purchase
df_corr = df.copy()
df_corr['Stay_In_Current_City_Years'] = df_corr['Stay_In_Current_City_Years'].replace('4+', '4').astype(int)

corr_occ, p_occ = stats.pearsonr(df_corr['Occupation'], df_corr['Purchase'])
corr_stay, p_stay = stats.pearsonr(df_corr['Stay_In_Current_City_Years'], df_corr['Purchase'])

print(f"Correlation Occupation vs Purchase: r={corr_occ:.4f}, p={p_occ:.4e}")
print(f"Correlation City Tenure vs Purchase: r={corr_stay:.4f}, p={p_stay:.4e}")

Correlation Occupation vs Purchase: r=0.0208, p=7.2597e-54
Correlation City Tenure vs Purchase: r=0.0054, p=5.7816e-05

```

